

SYSTEM DOCUMENTATION

Solutions Platform

Internal operations system for the Solutions agency — the current build, and the plan for what comes next.

DOCUMENT

Current state & roadmap

AUDIENCE

Solutions team & developers

BACKEND

NestJS 11 · PostgreSQL

This document describes what has been verified in the codebase, not what was planned. Where a feature is incomplete or missing it is marked as such, including the gaps that block production deployment. Section 7 lists those gaps in full.

– Contents

PART I · THE SYSTEM TODAY

- 1 What the platform is
 - 2 Architecture and stack
 - 3 People, roles and permissions
 - 4 Feature inventory
 - 5 The content approval workflow
 - 6 API reference
 - 7 Known gaps
-

PART II · WHAT WE DO NEXT

- 8 Why the flow feels wrong
 - 9 The plan, by priority
 - 10 Sequence and dependencies
 - 11 Decisions we need
-

APPENDIX

- A Running the system
 - B Environment variables
 - C Enumerated values
-

The system today

Nineteen modules and roughly sixty endpoints are built and working. The difficult foundations — multi-client isolation, role enforcement, and the approval state machine — are real implementations, not scaffolding. The weaknesses are in the connective tissue between features, and in the operational pieces needed to run in production.

1 What the platform is

Solutions is the system itself. It is not a record in the database. The records are the **clients** Solutions works for — Al Zaman, Curby, Taxero.

An employee works across several clients, holds a possibly different role on each, and sees only the clients they are assigned to. The interface therefore reads **Solutions (Curby)**: Solutions is the product, the client is the workspace currently in view.

BUSINESS LANGUAGE MAPPED TO THE DATA MODEL

Business term	In the code	Meaning
Solutions	—	The application. No table, no endpoint
Client	<code>company</code>	Al Zaman, Curby, Taxero — one row each
Employee	<code>user</code>	A member of Solutions staff
"Dina works on Curby as Designer"	<code>company_membership</code>	Links an employee to a client, carrying the role
The clients I work on	<code>GET /companies</code>	Returns only that employee's clients

NAMING NOTE

The API calls a client a `company`, and every feature route is scoped `/api/companies/{companyId}/...`. The interface says "client". This split is deliberate for now — renaming the routes is a large refactor and the permission rules are still settling — but it should be unified once they are final.

Data isolation

Every feature route carries a `companyId`, and two guards run on it. `CompanyAccessGuard` confirms the caller has an active membership for that client; `CompanyRolesGuard` then confirms their role on *that* client permits the action. An employee on Curby cannot read Taxero data by changing the URL. This is the property the whole system depends on and it should be re-verified after any change to routing or guards.

2 Architecture and stack

BACKEND

Concern	Choice
Framework	NestJS 11 (modular, one module per domain)

Concern	Choice
Database	PostgreSQL via TypeORM
Authentication	JWT access token + refresh token in an httpOnly cookie
Password hashing	argon2id
File storage	S3-compatible (MinIO locally), presigned download URLs
Validation	class-validator on every request body; Joi on environment variables
Rate limiting	Global throttler, tightened on authentication routes
AI	OpenAI SDK, with a mock provider used by default
Hardening	helmet, compression, CORS allow-list, strict body whitelisting

Requests are validated with `whitelist` and `forbidNonWhitelisted` enabled, so any undeclared field in a request body is rejected with a 400 rather than silently ignored.

3 People, roles and permissions

Authorisation works on two independent levels. A **platform role** describes standing within Solutions. A **client role** describes what a person does on one specific client, and the same person may hold different client roles on different clients.

PLATFORM ROLES

Role	Scope
SUPER_ADMIN	Full access. Bypasses client role checks on every client
AGENCY_ADMIN	Solutions management. Creates clients, creates employees, staffs clients
USER	Normal employee. Access is entirely determined by their client memberships

CLIENT ROLES

Role	Responsibility
ACCOUNT_MANAGER	Runs the client. The only client role that can invite, report and manage automations
SOCIAL_MEDIA_MANAGER	Content plans, posts, publishing
COPYWRITER	Captions and copy tasks
DESIGNER	Visual assets, file uploads
SALES_AGENT	The leads pipeline
CLIENT_OWNER	Client side — approves work, reads reports
CLIENT_REVIEWER	Client side — approves work

The last two are not Solutions employees. They are the client's own people, and they reach the system by invitation.

CAPABILITY MATRIX – CLIENT ROLES

Capability	AM	SMM	Copy	Design	Sales	Owner	Rev.
------------	----	-----	------	--------	-------	-------	------

Capability	AM	SMM	Copy	Design	Sales	Owner	Rev.
Manage members & invitations	●	—	—	—	—	—	—
Create / edit content plans	●	●	—	—	—	—	—
Create / edit posts	●	●	—	—	—	—	—
Submit to client · publish	●	●	—	—	—	—	—
Approve · request changes · reject	●	—	—	—	—	●	●
Comment on posts	●	●	●	●	—	●	●
Upload files & post assets	●	●	—	●	—	—	—
Create / edit leads	●	—	—	—	●	●	—
Create / assign tasks	●	●	—	—	—	—	—
Work on tasks	●	●	●	●	●	—	—
Manage campaigns	●	●	—	—	—	—	—
Manage responsibility matrix	●	—	—	—	—	—	—
Reports overview	●	—	—	—	—	●	—
Generate monthly report	●	—	—	—	—	—	—
Automation rules	●	—	—	—	—	—	—

Reading is broader than writing. The brand profile, and viewing content plans, posts, leads, tasks, campaigns and the responsibility matrix, are open to every active member of a client.

TWO READ PERMISSIONS TO REVIEW

Any member can **read the whole leads pipeline**, so a designer or copywriter can see the client's sales activity. Separately, **any member can edit the brand profile**, including client-side reviewers. Neither is likely intended; both are single-line fixes once you decide the rule.

4 Feature inventory

Verified against the source, module by module.

BUILT AND WORKING

Area	State	What exists
Clients & staffing	SOLID	Clients, employees, per-client role assignment, isolation between clients
Authentication	SOLID	Login, refresh, logout, current user. Deactivating a user ends their session immediately
Invitations	SOLID	Hashed single-use tokens, 7-day expiry, accept flow creates the account and membership
Brand profile	SOLID	Identity, audience, tone, languages, colours, services, offers, forbidden words. Feeds the AI

Area	State	What exists
Content plans	SOLID	Monthly plans with goals and a five-stage status flow
Content posts	SOLID	Six platforms, five content types, captions, visual briefs, scheduling fields
Approval workflow	SOLID	Eight statuses with enforced transitions, threaded comments, complete per-post audit log
Files & assets	SOLID	Real S3/MinIO uploads to 20 MB, presigned downloads, five asset types attachable to posts
Leads	SOLID	Seven statuses, nine sources, assignment, notes, full status history, follow-up dates
Tasks	SOLID	Seven types, four priorities, comments, file attachments, activity log, "my tasks"
Campaigns	SOLID	Six objectives, budget, dates, links posts + leads + tasks, overview rollup
Responsibility matrix	SOLID	Areas, eleven responsibility types, bulk assignment, matrix view
Reports	SOLID	Computes real figures: posts by status/platform/type, leads by status/source, conversion rate, saved monthly reports

PARTIALLY BUILT

Area	State	What is missing
Notifications	IN-APP ONLY	Fifteen event types, correctly targeted by role, unread counts. But there is no email or push, so a notification is only ever seen by someone already logged in
AI assistance	MOCK DEFAULT	Complete generate → review → apply pipeline with saved history: content plan previews, post ideas, captions. Runs on a mock provider unless AI_PROVIDER is switched to openai
Automations	REAL, NARROW	Genuinely wired to live events with per-client rules and run history — but only two triggers exist (changes requested; lead became interested) and there is no interface for managing rules

5 The content approval workflow

The most intricate logic in the system, and the piece most worth protecting with tests. Transitions are enforced server-side; any move not listed here is rejected.

PERMITTED TRANSITIONS

Action	Allowed from	Results in	Who may act
Submit to client	DRAFT · IN_INTERNAL_REVIEW · CHANGES_REQUESTED · APPROVED	READY_FOR_CLIENT	Account Manager, Social Media Manager
Approve	READY_FOR_CLIENT · CHANGES_REQUESTED	APPROVED	Client Owner, Client Reviewer, Account Manager
Request changes	READY_FOR_CLIENT · APPROVED	CHANGES_REQUESTED	Client Owner, Client Reviewer, Account Manager
Reject	READY_FOR_CLIENT · CHANGES_REQUESTED · APPROVED	CANCELED	Client Owner, Client Reviewer, Account Manager

Action	Allowed from	Results in	Who may act
Publish URL REQUIRED	APPROVED · SCHEDULED	PUBLISHED	Account Manager, Social Media Manager

Requesting changes also fires the automation engine, which can create fix-up tasks for the copywriter and designer. Every transition writes to the post's approval log with actor, timestamp and note, and notifies the roles that need to know.

THE GAP IN THIS FLOW

SCHEDULED is a valid status and posts carry a `scheduledAt` date, but **no scheduler exists in the codebase**. Nothing fires when that time arrives. Publishing means posting on the platform by hand and pasting the resulting URL back in — the system records publication, it does not perform it.

6 API reference

Base URL `/api`. All routes except login, token refresh and invitation acceptance require `Authorization: Bearer <accessToken>`. Feature routes are scoped `/companies/{companyId}/...`.

ENDPOINTS BY AREA

Area	Routes
Authentication	POST <code>/auth/login</code> · <code>/auth/refresh</code> · <code>/auth/logout</code> · GET <code>/auth/me</code> · POST <code>/auth/accept-invitation</code>
Employees (<i>admin</i>)	POST <code>/users</code> · GET <code>/users</code> · GET <code>/users/{userId}</code> · PATCH <code>/users/{userId}</code>
Clients	GET <code>/companies</code> · POST <code>/companies</code> (<i>admin</i>) · GET, PATCH <code>/companies/{id}</code>
Members	POST <code>/members</code> (<i>admin</i>) · GET <code>/members</code> · PATCH, DELETE <code>/members/{membershipId}</code>
Invitations	POST <code>/invitations</code> · GET <code>/invitations</code>
Brand profile	POST, GET, PATCH <code>/brand-profile</code>
Content plans	POST, GET <code>/content-plans</code> · GET, PATCH <code>/content-plans/{id}</code>
Posts	POST, GET <code>/posts</code> · GET, PATCH <code>/posts/{id}</code>
Approvals	POST <code>/posts/{id}/submit-review</code> · <code>/approve</code> · <code>/request-changes</code> · <code>/reject</code> · <code>/publish</code> · POST, GET <code>/posts/{id}/comments</code> · GET <code>/posts/{id}/approval-logs</code>
Files	POST <code>/files/upload</code> (multipart, field <code>file</code> , ≤20 MB) · GET <code>/files/{id}</code> · GET <code>/files/{id}/download-url</code>
Post assets	POST, GET <code>/posts/{id}/assets</code> · DELETE <code>/posts/{id}/assets/{assetId}</code>
AI	POST <code>/ai/content-plan-preview</code> · <code>/ai/post-ideas</code> · <code>/ai/caption</code> · GET <code>/ai/generations</code> · POST <code>/ai/generations/{id}/apply-content-plan</code> <code>apply-caption</code> <code>apply-post-idea</code>
Leads	POST, GET <code>/leads</code> · GET, PATCH <code>/leads/{id}</code> · PATCH <code>/leads/{id}/status</code> · POST, GET <code>/leads/{id}/notes</code> · GET <code>/leads/{id}/status-history</code>
Tasks	POST, GET <code>/tasks</code> · GET <code>/tasks/my</code> · GET, PATCH <code>/tasks/{id}</code> · PATCH <code>/tasks/{id}/status</code> · POST, GET <code>/tasks/{id}/comments</code> · POST, GET <code>/tasks/{id}/attachments</code> · DELETE <code>/tasks/{id}/attachments/{attachmentId}</code> · GET <code>/tasks/{id}/activity-logs</code>
Campaigns	POST, GET <code>/campaigns</code> · GET, PATCH <code>/campaigns/{id}</code> · PATCH <code>/campaigns/{id}/status</code> · GET <code>/campaigns/{id}/overview</code> · POST, DELETE <code>/campaigns/{id}/posts</code> <code>leads</code> <code>tasks</code> <code>{entityId}</code>

Area	Routes
Responsibilities	POST, GET /responsibility-areas · GET, PATCH, DELETE /responsibility-areas/{id} · POST /responsibility-assignments · POST /responsibility-assignments/bulk · GET /responsibility-assignments/matrix · GET, PATCH, DELETE /responsibility-assignments/{id}
Reports	GET /reports/overview?month=&year= · POST /reports/monthly · GET /reports · GET /reports/{id}
Notifications	GET /notifications · GET /notifications/unread-count · PATCH /notifications/read-all · PATCH /notifications/{id}/read
Automations	POST, GET /automation-rules · GET, PATCH /automation-rules/{id} · GET /automation-runs · GET /automation-runs/{id}

CONVENTIONS

- Passwords must be at least 12 characters wherever one is set.
- Undeclared fields in a request body produce a 400 — send only documented fields.
- Rate limits: 5/min on login, 20/min on refresh, 100/min elsewhere.
- Enum values are exact strings; anything outside the documented set is a 400.
- There is **no** registration endpoint. Accounts are created by administrators, or by accepting an invitation.

7 Known gaps

Stated plainly, because these determine what can be tested and what can be deployed.

GAPS, BY CONSEQUENCE

Gap	Blocks	Detail
No email of any kind	TESTING	Nothing in the codebase sends mail. Invitation tokens are returned in an API response and must be carried to the recipient by hand. Admin-created employees receive a temporary password the same way
Invitation link path mismatch	TESTING	The API returns /auth/accept-invitation?token=... while the frontend route is /accept-invitation, so generated invite links reach the 404 page. One line, on either side
No database migrations	PRODUCTION	Schema is created by TypeORM synchronize, which is enabled only in development. A production deployment would start against an empty database with no way to create its tables
No scheduler	FEATURES	No cron or job runner exists. Scheduled publishing, follow-up reminders and overdue alerts are therefore impossible today
Essentially no tests	CONFIDENCE	Only the default framework example files. The approval state machine and the client isolation guards — the two places a bug is most costly — are unprotected
Audit module is empty	ACCOUNTABILITY	The folder exists with no code. Per-post and per-task histories do exist, but there is no system-wide audit trail
No pagination on posts	SCALE	Leads, tasks, campaigns, notifications and responsibilities all accept a limit. Posts do not, and will degrade once a client has a year of content
Redis required but unused	HYGIENE	REDIS_HOST is a required environment variable and the container runs, but no code uses it and no Redis client is even installed. It will refuse to start without a service it never contacts

ONE SILVER LINING

Because Redis is already provisioned and running, the infrastructure needed for scheduled jobs, distributed locks and background queues is in place. It simply has not been used yet.

What we do next

The system has not been tested end to end, and the reason is structural rather than a matter of effort: onboarding a person requires hand-carrying a secret, and the daily flow asks people to type in work the system already knows about. This part diagnoses that, then sets out a plan in priority order.

8 Why the flow feels wrong

1 Nobody can get into the system

Invitations return a token in an API response, so an administrator must copy it out of a dialog and send it over a messaging app — and the resulting link is broken anyway. There is no email anywhere in the codebase. This single gap blocks testing the approval workflow, because approvals require a client contact who currently cannot be onboarded.

2 It is a set of forms, not a workflow

Creating a content plan does not create posts. Creating a post does not create tasks for the designer and the copywriter. Every step re-enters information the system already holds. That is the source of the "bad flow" feeling: staff feed the database instead of the database feeding them work.

3 There is no "what do I do today"

An employee signs in and arrives at a dashboard rather than at their work. Nothing assembles *my tasks, posts awaiting my action, and leads due for follow-up* across the clients that person works on. The API shape prevents it: every route is scoped to one client, so the question cannot even be asked.

4 Clients must use an internal tool

To approve one post, the contact at Al Zaman needs an account, an invitation, a password, and then has to navigate an agency dashboard built for staff. That friction is the usual reason client approvals stall.

5 "Scheduled" does not schedule

A post can be given a date and moved to `SCHEDULED`, but no job ever runs. The system is entirely passive: it never tells anyone that something is due, overdue, or waiting on them.

6 The responsibility matrix is decorative

This is the feature closest to the original goal — rules and responsibility per employee — and nothing reads it. Recording that "Dina manages Instagram content" changes no behaviour anywhere in the system. The structural reason is that responsibility areas have no link to work items, so nothing can be routed by them.

9 The plan, by priority

P0 — Before anything can be tested

1 Replace passwords-in-transit with activation links

Rather than bolting email onto the current design, collapse the two onboarding paths into one. An administrator creates an employee with no password at all; the invited client contact follows the same route. Both receive one email containing a "set your password" link. The token machinery already exists in the invitations module and only needs generalising.

Use SMTP through the existing Solutions mailbox — no third-party service required. When SMTP is not configured, write the activation link to the log, so development and testing never block on mail delivery again.

Effort **1-2 days** · Unlocks **all end-to-end testing**

2 Fix the invitation link path

Align the path the API returns with the route the frontend registers.

Effort **minutes** · Unlocks **the entire approval workflow**

3 Write a demo seed script

Two clients, six employees with mixed roles, a month of posts spread across every status, twenty leads, and a set of open tasks. A flow cannot be judged against an empty database, and hand-entering test data is precisely why testing keeps being postponed.

Effort **1 day** · Unlocks **meaningful review sessions**

P1 — Turn it into a system

4 A cross-client "my work" endpoint

Introduce a `/api/me/...` namespace that is not scoped to a single client — the one architectural change on this list. `GET /api/me/work` returns, grouped by client and ordered by urgency: tasks assigned to me with overdue first; posts awaiting my action, resolved from my role on each client; leads whose follow-up date has arrived; and unread notifications.

This becomes the landing page and is the largest single improvement in how the product feels. Staff open the application and see their day rather than a menu.

Effort **1-2 days** · Unlocks **the home screen for everything after**

5 Generate work automatically, through the existing engine

Do not hard-code "a post creates tasks". The automation module already stores per-client trigger-and-action rules with a free-form config column; it simply has only two triggers. Add `POST_CREATED →`

`CREATE_CONTENT_TASKS`, `CONTENT_PLAN_APPROVED → CREATE_POSTS_FROM_PLAN` and `POST_APPROVED → CREATE_PUBLISHING_TASK`.

The rule config holds the template — for a reel, a script task for the copywriter and an edit task for the designer, each due a set number of days before the scheduled date. Keeping this in rules rather than in code matters because clients differ: one may require client review on every post, another may not.

Effort **2 days** · Unlocks **the system generating work instead of receiving it**

6 Make the responsibility matrix operational

Add an optional responsibility-area reference to posts and tasks. That one field turns the matrix from a chart into a routing table.

Matrix records	System behaviour
TO_WORK_ON	Receives the auto-created task, replacing role-based guesswork
TO_APPROVE	Receives the approval request — not everyone holding the role
TO_SUPERVISE · TO_BE_HELD_RESPONSIBLE	Notified when work in that area runs late
TO_BE_INFORMED	Receives a digest of changes in that area
TO_CONSULT	Suggested when a task is blocked or questioned

Then surface it where the work is — showing "Instagram content — Dina manages, Amir approves" on the post itself. The matrix currently lives on a page nobody opens; placed beside the work it becomes the answer to "who owns this?"

Effort **2–3 days** · Unlocks **the original goal, actually operating**

7 Public client review links

A revocable, expiring tokenised URL that shows a post — title, caption, images via presigned links — and accepts approve or request-changes with a name and a note, without any account. It writes into the same approval log, attributed as "via review link", so the audit trail stays whole.

Extend the same mechanism to a whole content plan, so a client can approve a month in one sitting. That is what agencies actually want, and it is the version clients will praise.

Trade-off: weaker identity than a login. Mitigate with expiry, revocation, and binding the link to the invited email address.

Effort **2–3 days** · Unlocks **approvals that do not depend on client software adoption**

P2 — Before production

8 Database migrations

Non-negotiable. Today a production deployment creates no tables at all.

Effort **1 day** · Unlocks **deployment**

9 Tests on the two costly areas

The approval state machine and client data isolation. Everything else can be caught by hand; a silent isolation failure cannot.

Effort **2 days** · Unlocks **changing code without fear**

10 Housekeeping

Pagination on posts; either use Redis or stop requiring it; fill in the audit module; settle the two broad read permissions noted in section 3.

Effort **1–2 days**

P3 — Leverage, once the loop works

11 A scheduler, in two stages

Now: a job every few minutes that reads the clock and nudges — an approved post whose date has arrived notifies the social media manager and creates a publishing task; a lead due for follow-up notifies its owner; an overdue task notifies the assignee and their supervisor from the matrix. This changes the system's character from passive to proactive without touching a single social platform. It is also where the idle Redis earns its place, providing the lock that stops jobs firing twice across instances.

Later: real publishing through the Meta and LinkedIn APIs — per-client authorisation, token refresh, media upload and platform-specific rules. Substantial work that should not block anything above it.

Stage one 1 day · Stage two weeks

12 Further candidates

A content calendar as a month grid, which is the view agencies live in. Workload per employee across clients, using data already held. Recurring tasks for retainer work. Design file versioning, so revisions are traceable. A client-facing dashboard. Real AI generation, by switching the provider that is already integrated.

10 Sequence and dependencies

Several items depend on each other, so order matters more than parallel effort.

SUGGESTED ORDER

Stage	Work	Why here
WEEK 1	Activation emails · link path fix · demo seed script	Without these, testing keeps being deferred. Nothing else should start first
WEEK 2	"My work" endpoint, then automation triggers	The work view comes first so there is somewhere to see the work the triggers create
WEEK 3	Matrix routing, then the nudge scheduler	Routing needs generated tasks to route; nudges need something worth nudging about
THEN	Migrations and tests, then public review links	Required before any real deployment
LATER	Real social publishing, calendar, workload	High value, but each is large and none is blocking

11 Decisions we need

OPEN QUESTIONS

Question	Why it matters	Recommendation
Which side fixes the invitation path?	Blocks every invitation today	Change the backend to match whatever route the frontend already has — less churn
Should leads be readable by everyone?	Designers and copywriters currently see the full sales pipeline	Restrict reading to Account Manager, Sales Agent and Client Owner
Who may edit the brand profile?	Client-side reviewers can currently change it	Restrict to Account Manager and Social Media Manager

Question	Why it matters	Recommendation
Turn on real AI?	The integration is built; only the provider setting and an API key are needed	Switch it on in a test environment once the brand profiles hold real content
How far should publishing go?	Determines whether stage two of the scheduler is ever needed	Decide after the nudge scheduler is in use — reminders may prove sufficient
Rename <code>company</code> to <code>client</code> ?	The API and the interface currently disagree	Do it once the permission rules are final, in one deliberate pass

Reference

Operational detail for whoever runs or extends the system.

A Running the system

```
# 1 - Infrastructure, from solutions-platform/  
docker compose up -d postgres redis minio  
  
# 2 - API, from solutions-platform/api/  
npm ci  
npm run start:dev          # listens on port 3000, prefix /api  
  
# 3 - Frontend  
npm ci  
npm run dev                # http://localhost:5173
```

The API will not start while MinIO is unavailable, because it verifies its storage bucket during boot. It also refuses to start without `REDIS_HOST`, despite never contacting Redis.

The first administrator is created on boot from `BOOTSTRAP_ADMIN_EMAIL` and `BOOTSTRAP_ADMIN_PASSWORD`. This is required because there is no public sign-up: without it, no one could sign in and no one could create anyone. The step is idempotent — it creates the account when missing, promotes it if it exists without administrator rights, and never overwrites an existing password.

B Environment variables

Variable	Purpose
<code>NODE_ENV</code> · <code>PORT</code>	Runtime mode and port. Schema auto-sync happens only in <code>development</code>
<code>DATABASE_HOST</code> · <code>_PORT</code> · <code>_NAME</code> · <code>_USER</code> · <code>_PASSWORD</code>	PostgreSQL connection
<code>JWT_ACCESS_SECRET</code> · <code>JWT_REFRESH_SECRET</code>	Token signing. Minimum 32 characters each
<code>JWT_ACCESS_EXPIRES_IN</code> · <code>JWT_REFRESH_EXPIRES_IN</code>	Token lifetimes. Default 15 minutes and 30 days
<code>REFRESH_TOKEN_COOKIE_NAME</code>	Refresh cookie name, scoped to path <code>/api/auth</code>
<code>BOOTSTRAP_ADMIN_EMAIL</code> · <code>_PASSWORD</code> · <code>_NAME</code>	First administrator, seeded on boot
<code>FRONTEND_URL</code>	CORS allow-list. Accepts a comma-separated list
<code>S3_ENDPOINT</code> · <code>_ACCESS_KEY</code> · <code>_SECRET_KEY</code> · <code>_BUCKET</code> · <code>_REGION</code> · <code>_FORCE_PATH_STYLE</code>	Object storage
<code>AI_PROVIDER</code> · <code>AI_API_KEY</code> · <code>AI_MODEL</code>	<code>mock</code> or <code>openai</code> . A key is required only for the latter
<code>REDIS_HOST</code> · <code>REDIS_PORT</code>	Required, but currently unused by any code

c Enumerated values

Set	Values
Platform role	USER · AGENCY_ADMIN · SUPER_ADMIN
User status	ACTIVE · INACTIVE · SUSPENDED
Client status	ACTIVE · INACTIVE · ARCHIVED
Client role	ACCOUNT_MANAGER · COPYWRITER · DESIGNER · SOCIAL_MEDIA_MANAGER · CLIENT_OWNER · CLIENT_REVIEWER · SALES_AGENT
Membership status	ACTIVE · INVITED · SUSPENDED
Invitation status	PENDING · ACCEPTED · REVOKED · EXPIRED
Content plan status	DRAFT · INTERNAL_REVIEW · CLIENT_REVIEW · APPROVED · ARCHIVED
Post status	DRAFT · IN_INTERNAL_REVIEW · READY_FOR_CLIENT · CHANGES_REQUESTED · APPROVED · SCHEDULED · PUBLISHED · CANCELED
Platform	INSTAGRAM · FACEBOOK · TIKTOK · LINKEDIN · WHATSAPP · WEBSITE
Content type	POST · REEL · STORY · CAROUSEL · VIDEO
Post asset type	DESIGN · REFERENCE · VIDEO · THUMBNAIL · DOCUMENT
Lead status	NEW · CONTACTED · INTERESTED · WAITING_DECISION · WON · LOST · FOLLOW_UP_LATER
Lead source	FACEBOOK · INSTAGRAM · WHATSAPP · WEBSITE · MANUAL · REFERRAL · TIKTOK · LINKEDIN · OTHER
Task status	TODO · IN_PROGRESS · IN_REVIEW · BLOCKED · DONE · CANCELED
Task type	COPYWRITING · DESIGN · CLIENT_REVIEW · FOLLOW_UP · PUBLISHING · REPORTING · GENERAL
Task priority	LOW · MEDIUM · HIGH · URGENT
Campaign objective	AWARENESS · ENGAGEMENT · LEADS · SALES · RETENTION · LAUNCH
Campaign status	DRAFT · ACTIVE · PAUSED · COMPLETED · CANCELED
Responsibility type	TO_MANAGE · TO_SUPPORT · TO_FOLLOW_UP · TO_BE_HELD_RESPONSIBLE · TO_APPROVE · TO_WORK_ON · TO_CONSULT · TO_COORDINATE · TO_BE_INFORMED · TO_SUPERVISE · OTHER
Notification type	TASK_ASSIGNED · TASK_STATUS_CHANGED · TASK_COMMENTED · POST_SUBMITTED_TO_CLIENT · POST_CHANGES_REQUESTED · POST_APPROVED · POST_PUBLISHED · POST_COMMENTED · POST_REJECTED · LEAD_ASSIGNED · LEAD_STATUS_CHANGED · LEAD_NOTE_ADDED · INVITATION_CREATED · INVITATION_ACCEPTED · REPORT_CREATED
Automation trigger	POST_CHANGES_REQUESTED · LEAD_BECAME_INTERESTED
Automation action	CREATE_POST_CHANGES_TASKS · CREATE_LEAD_FOLLOW_UP_TASK

Compiled from the state of the `api` repository. Every feature, endpoint, permission and gap listed here was read from source rather than from prior planning documents. Behaviour against a live database has not been exercised end to end — see Part II.